

High Performance Data Analytics and Visualisation

Assignment: Housing Data Visual - Report

Matteo Arrigo

October 31, 2025

1 Introduction

This report presents an interactive visualization system for housing market data, combining a 2D Scatterplot and a Parallel Coordinates Plot (PCP) for multivariate analysis. The focus is on the PCP and coordinated brushing between views.

The considered dataset has 545 records and 13 attributes: continuous (e.g., price, area), discrete (e.g., bedrooms), and categorical (e.g., airconditioning). Each type requires different visual encoding.

2 Visual Encoding Design

2.1 Scatterplot (Brief Overview)

The scatterplot maps area (x) and price (y), using color to show selection. It helps explore price-area correlation and outliers.

2.2 Parallel Coordinates Plot (PCP) Design

The PCP is designed to visualize all 13 attributes simultaneously, facilitating the exploration of complex relationships in high-dimensional data.

2.2.1 Multi-Type Scale System

The PCP supports all 13 attributes, using linear scales for continuous, and point scales for discrete/categorical data. Users can add/remove axes, reorder them, and swap dimensions via dropdowns. Line color and opacity indicate selection. The 60% width allows at least 6 axes to be shown clearly.

2.2.2 Interactive Dimension Control

To manage 13 dimensions without visual clutter, the PCP provides three complementary interaction mechanisms:

- **Dynamic axis count control** via a number input (range 2-13, default 6) positioned below the visualization. It allows users to add or remove axes on demand: when axes are added, dimensions are selected sequentially from the full attribute list; when removed, the rightmost axes are eliminated.
- **Dropdown menus** on each axis enable selection from all 13 available dimensions for flexible dimension substitution. The implementation embeds HTML dropdowns within SVG using `foreignObject` elements
- **Drag-and-drop reordering** via drag handles above each dropdown, implemented using D3's drag behavior. Indeed, reordering is crucial because PCP patterns depend heavily on axis adjacency.

2.2.3 Visual Encoding Strategy

Like the classical PCP seen during the lectures, position encoding is used for both dimensions (X-axis, for the different dimensions to show) and values (Y-axis, quantitative/categorical per dimension). Line color, opacity and stroke width indicates selection state. In particular, low opacity for unselected lines reduces clutter with 545 polylines.

The 60% width allocation provides adequate space for at least 6 axes while maintaining readability.

State	Color	Opacity	Stroke width
Unselected	#B0BEC5 (grey)	0.2	1.2px
Selected	#C62828 (red)	0.8	1.8px
Highlighted (hover)	#F9A825 (yellow)	1	3px

2.3 Coordinated Brushing Interaction

Brushing in either view highlights corresponding data in both. Scatterplot uses a 2D brush; PCP uses 1D brushes per axis. Clicking selects individual items. Brushing in one view clears the other for clarity.

Also, both views support single-item selection via click. In the scatterplot, clicking a point toggles its membership in the current selection (it is added or removed) without replacing other selected items. In the PCP, where line overlap is common due to the high number of discrete values, clicking a polyline defines a new filter: the values at the line’s terminal axes (left and right dimensions of the selected segment) are used to select all matching records, and this action replaces any prior selection.

3 Advantages and Limitations

3.1 Parallel Coordinates Strengths

The PCP enables multivariate analysis and flexible filtering, but can be cluttered with many polylines. Axis order and type affect readability. Other approaches (e.g., pairwise scatterplots) were considered but are less scalable. PCP with brushing offers the best trade-off for this dataset.

3.2 Parallel Coordinates Limitations

Despite opacity adjustments, rendering 545 polylines still generates substantial visual clutter: numerous discrete values produce heavy line overlap that can obscure multivariate patterns. Readability is highly sensitive to axis ordering and to the fact that each dimension uses its own y-scale, which makes PCPs harder to interpret than simple scatterplots. Mitigations implemented include interactive brushing to reduce noise, drag-and-drop axis reordering to expose adjacency-dependent patterns, and per-axis dropdowns to substitute or aggregate dimensions for targeted exploration.

3.3 Alternative Approaches Considered

Several alternative visualizations were considered but not adopted due to scalability or fidelity trade-offs.

For example, a pairwise scatterplot exposes bivariate relationships but, with 13 attributes, would produce 78 cells and severe overplotting for 545 records; interactive enhancements mitigate this somewhat but remain cumbersome for high-dimensional exploration. Instead, tree- or aggregation-based layouts reveal group structure but abstract away the per-record detail required for axis-based brushing, so they were deprioritized.

In conclusion, I found that the PCP with coordinated brushing best balances per-record multivariate comparison and interactive exploration for this dataset.

4 Implementation Patterns and Design Choices

4.1 React Component Architecture and State Management

The system uses React for state and D3 for rendering, with clear separation. Four main components manage state and visualization. Brushing and selection are coordinated via React hooks and callbacks. Responsive sizing and robust CSV parsing are implemented.

Inter-component communication is achieved through state lifting and callback registration patterns. A complete example of this is the coordinated brushing interaction.

4.1.1 Coordinated brushing

Cross-view brush coordination is achieved through React hooks (`useRef`, and `useEffect`). Each visualization container exposes its brush-clearing function by assigning it to a ref provided by the parent component during mount. The parent component maintains refs to both views’ clear functions (`scatterplotClearBrushRef`, `parallelCoordinatesClearBrushRef`).

Controller method objects provide an interface between D3 events and React state. When a brush activates in one view, the D3 event handler calls a controller method that invokes the other view’s clear function via its

ref. For selection updates, D3 events trigger controller methods that update the parent’s `selectedItems` state via `setSelectedItems()`. This state change propagates back to both containers as props, where `useEffect` hooks detect the update and call D3 highlighting methods. This maintains React’s unidirectional data flow while enabling coordinated interaction through a simple ref-based communication pattern. To better understand this, a sequence diagram is provided in the repository, inside the `report/` folder.

4.2 React and D3 Separation of Responsibilities

The implementation maintains strict separation between React (state management) and D3 (visualization rendering). React containers handle lifecycle management via `useEffect` hooks for component mounting / unmounting and D3 instance creation/cleanup. The `useRef` hook persists D3 instances across renders. Controller method objects bridge D3 callbacks to React state updates without D3 directly modifying state.

D3 classes own all SVG creation, DOM manipulation, scales, and axes. Rendering logic uses data binding with `.data()`, `.join()`, and `.enter()` patterns for efficient enter/update/exit handling. D3 event listeners (drag, brush, click) callback to React via provided controller methods. This architecture ensures maintainable code.

4.3 Other Implementation Decisions

Brush ranges are stored with composite keys (`dimension_axisIndex`) to support dynamic axis reordering. All D3 callbacks route through controller method objects, maintaining unidirectional data flow (React → D3 → React callbacks). Responsive sizing uses container dimensions (`offsetWidth/Height`) rather than hard-coded values. CSV parsing includes empty row filtering to prevent null value issues discovered during development.

5 User Tasks Enabled by the Visualization

The visual system that was developed enables several common exploratory analysis tasks.

Below are representative tasks and concise how-to instructions using the provided interactions (brushing, axis reordering, dropdown substitution, and axis-count control).

- **Find outliers and leverage points:** In the scatterplot, drag a rectangular brush around points far from the main price–area trend. The corresponding polylines are highlighted in the PCP for multivariate inspection across all visible dimensions.
- **Multi-criteria filtering (AND across axes):** In the PCP, apply vertical brushes on multiple axes. Only records satisfying all active axis filters remain emphasized in both views.
- **Compare categorical segments against price:** Brush a category on an axis (e.g., `airconditioning = yes`). Drag axes so that `price` is adjacent to the brushed category to visually compare dispersion and central tendency between segments.
- **Investigate bivariate relationships:** Drag axes to place two dimensions side-by-side. Parallel bundles indicate positive association; frequent crossings indicate inverse or heterogeneous relationships.
- **Budget-to-spec search:** Constrain `price` (*max*) and `area` (*min*) with PCP brushes; optionally add constraints on `stories` and `parking`. Use the scatterplot to check geographic-like spread in the price–area space and click points to include/exclude individual homes from the current selection.

6 Conclusion

To sum up, the system balances exploratory power and usability through type-aware scales, flexible axes, and coordinated brushing, and React and D3 are cleanly separated for maintainability.

This approach enables users to flexibly explore complex, high-dimensional data and uncover patterns that would be difficult to detect with simpler visualizations. The coordinated interactions between views support both broad overviews and detailed, attribute-specific analysis. Future improvements could include additional aggregation or clustering features to further reduce visual clutter and enhance interpretability for even larger datasets.

7 AI Acknowledgment

AI tools (main Copilot and ChatGPT) were used to debug and refine some parts of the code, and for text suggestions. The first draft of this report was generated by ChatGPT. All design and validation are by the author.